

DSC 550 - Data Mining

Manoj K Kola

Week 6 - Term Project Milestone 1: Data Selection and EDA

1. Import Required Libraries

This code sets up the environment for data analysis and visualization using Python. First, we import **pandas**, a powerful library for handling and manipulating structured data. Pandas provides efficient data structures such as DataFrames and Series, which make it easier to load, clean, and analyze datasets.

Next, we import **numpy**, a widely used library for numerical computation in Python. It allows us to perform array operations, execute mathematical calculations, and support a range of statistical functions.

For data visualization, we import **matplotlib.pyplot**, the foundational Python library for creating plots, charts, and other graphical representations of data. It provides flexibility for building both simple and complex visualizations. To complement this, we also import **seaborn**, a library built on top of matplotlib that enables the creation of aesthetically pleasing and informative statistical graphics with minimal code.

Finally, we set a consistent visual style for all seaborn plots using `sns.set(style="whitegrid")`. This applies a clean white background with grid lines, enhancing readability and ensuring all plots have a uniform, professional appearance.

```
In [1]: # Import pandas for data loading and data manipulation
import pandas as pd

# Import numpy for numerical operations
import numpy as np

# Import matplotlib for creating basic plots and charts
import matplotlib.pyplot as plt

# Import seaborn for advanced and visually appealing data visualizations
import seaborn as sns

# Set a clean and consistent visual style for all seaborn plots
sns.set(style="whitegrid")
```

2. Loading and Previewing the Telco Customer Churn Dataset

The code below loads the **Telco Customer Churn** dataset into a pandas DataFrame using the `read_csv` function. This allows us to work with the dataset in a structured tabular format, enabling efficient data manipulation and analysis.

After loading the data, we use the `head()` function to display the first five rows of the dataset. This provides a quick overview of the data's structure, including the column names, data types, and sample values, which helps in understanding the dataset before performing further analysis.

```
In [37]: # Load the Telco Customer Churn dataset into a DataFrame
df = pd.read_csv("Telco_Customer_Churn.csv")

# Display the first 5 rows of the dataset to understand its structure
df.head()
```

```
Out[37]:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
0	7590-VHVEG	Female	0	Yes	No	1	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No
3	7795-CFOCW	Male	0	No	No	45	No	No
4	9237-HQITU	Female	0	No	No	2	Yes	No

5 rows × 21 columns



3. Exploring the Dataset

The first step in understanding a dataset is to get a sense of its size and structure. Using `df.shape`, we can display the number of rows and columns in the dataset. This helps quickly assess the size of the dataset and the number of features available for analysis.

Next, we use `df.info()` to generate a concise summary of the DataFrame. This provides important information about the dataset, including the column names, the number of non-null entries in each column, and the data types of each feature. It is particularly useful for identifying missing values and understanding the structure of the data.

Finally, `df.describe()` is used to generate summary statistics for all numerical columns in the dataset. This includes measures such as count, mean, standard deviation, minimum, and

maximum values. These statistics provide an overview of the distribution and range of the numerical features, helping to identify patterns, outliers, and potential issues before deeper analysis.

```
In [38]: # Display the number of rows and columns in the dataset  
# This helps understand the size of the data  
  
df.shape
```

Out[38]: (7043, 21)

```
In [39]: # Display a concise summary of the DataFrame  
# This shows the number of rows and columns, column names,  
# non-null counts, and data types for each column  
  
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 7043 entries, 0 to 7042  
Data columns (total 21 columns):  
#   Column                Non-Null Count  Dtype  
---  -  
0   customerID            7043 non-null   object  
1   gender                 7043 non-null   object  
2   SeniorCitizen          7043 non-null   int64  
3   Partner                7043 non-null   object  
4   Dependents             7043 non-null   object  
5   tenure                 7043 non-null   int64  
6   PhoneService           7043 non-null   object  
7   MultipleLines          7043 non-null   object  
8   InternetService        7043 non-null   object  
9   OnlineSecurity         7043 non-null   object  
10  OnlineBackup           7043 non-null   object  
11  DeviceProtection       7043 non-null   object  
12  TechSupport            7043 non-null   object  
13  StreamingTV            7043 non-null   object  
14  StreamingMovies        7043 non-null   object  
15  Contract               7043 non-null   object  
16  PaperlessBilling       7043 non-null   object  
17  PaymentMethod          7043 non-null   object  
18  MonthlyCharges         7043 non-null   float64  
19  TotalCharges           7043 non-null   object  
20  Churn                  7043 non-null   object  
dtypes: float64(1), int64(2), object(18)  
memory usage: 1.1+ MB
```

```
In [40]: # Generate summary statistics for all numerical columns in the dataset  
# This includes count, mean, standard deviation, minimum, and maximum values  
  
df.describe()
```

Out[40]:

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

4. Milestone 1 – Business Problem Narrative

Business Problem Statement

Customer churn is a critical issue for telecommunications companies, as acquiring new customers is significantly more expensive than retaining existing ones. Even a small reduction in churn can result in substantial revenue savings and improved customer lifetime value.

The objective of this project is to analyze customer behavior and identify key factors that contribute to customer churn in a telecom organization. By understanding these drivers, the company can proactively identify high-risk customers and implement targeted retention strategies such as personalized offers, contract upgrades, or service improvements.

The dataset used for this project is the Telco Customer Churn dataset obtained from Kaggle. It contains customer demographic information, account details, service subscriptions, billing data, and churn status. This data represents a realistic business scenario faced by subscription-based service providers.

The target variable for this analysis is the **Churn** column, which indicates whether a customer has left the company. Since churn is a binary outcome (Yes or No), this problem is well suited for a classification modeling approach.

In this milestone, exploratory data analysis (EDA) and graphical analysis are performed to understand patterns, trends, and relationships within the data. The insights gained from this analysis will guide feature selection and model development in future milestones. This project demonstrates how data mining techniques can be applied to solve a real-world business problem with practical value for decision-makers.

5. Data Cleaning - Converting and Checking Data for Missing Values

In this code, we first convert the **TotalCharges** column in our dataset to a numeric format using `pd.to_numeric()`. Any values that are invalid, blank, or cannot be converted to numbers are automatically set to **NaN** (Not a Number) by using the parameter `errors="coerce"`. This ensures that all entries in the column are either numeric or explicitly marked as missing, which is important for accurate calculations and analysis.

After the conversion, we check for missing values in the dataset using `df.isnull().sum()`. This operation counts the number of **NaN** values in each column, allowing us to quickly identify which columns require data cleaning or imputation before further analysis or modeling.

```
In [41]: # Convert the 'TotalCharges' column to numeric format
# Any invalid or blank values will be converted to NaN
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# Check the number of missing values in each column
# This helps identify columns that may need cleaning
df.isnull().sum()
```

```
Out[41]: customerID      0
gender      0
SeniorCitizen  0
Partner      0
Dependents    0
tenure      0
PhoneService  0
MultipleLines  0
InternetService  0
OnlineSecurity  0
OnlineBackup  0
DeviceProtection  0
TechSupport   0
StreamingTV   0
StreamingMovies  0
Contract      0
PaperlessBilling  0
PaymentMethod  0
MonthlyCharges  0
TotalCharges  11
Churn         0
dtype: int64
```

6. Graphical Analysis (Minimum 4 Graphs)

Graph 1: Customer Churn Distribution Plot

The code below creates a visualization to show the distribution of customers by churn status. First, a new figure is initialized using `plt.figure()` to ensure that the plot is created on a fresh canvas.

We then use Seaborn's `countplot` function to display the number of customers in each churn category ("Yes" or "No"). The `color` parameter is set to `'seagreen'` to give the bars a distinct and visually appealing color. This type of plot is useful for quickly comparing the frequency of categorical values.

To make the plot more informative, we add a title `"Customer Churn Distribution"` using `plt.title()`. The x-axis is labeled `"Churn"` to indicate the churn status, while the y-axis is labeled `"Number of Customers"` to show the count for each category.

Finally, `plt.show()` is called to render and display the plot. This visual representation helps understand how many customers stayed versus churned, providing an overview of churn behavior in the dataset.

```
In [9]: # Create a new figure for the plot
plt.figure()

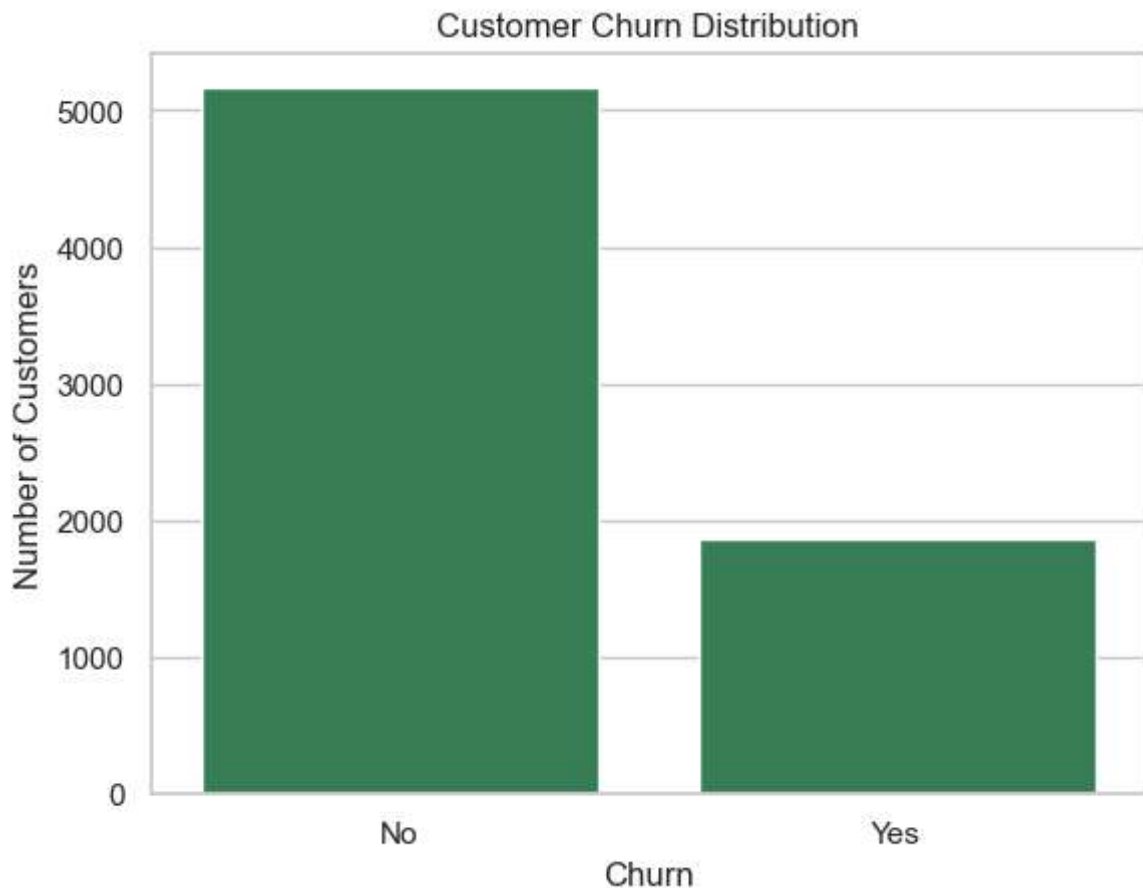
# Plot the count of customers for each churn category (Yes / No)
sns.countplot(x="Churn", data=df, color='seagreen' )

# Add a title to explain what the chart represents
plt.title("Customer Churn Distribution")

# Label the x-axis to indicate churn status
plt.xlabel("Churn")

# Label the y-axis to show the number of customers
plt.ylabel("Number of Customers")

# Display the plot
plt.show()
```



Insight: The dataset is imbalanced, with a higher number of customers not churning. This is important to consider during model evaluation.

Graph 2: Boxplot: Customer Tenure vs Churn

The following code creates a boxplot to visualize the distribution of customer tenure by churn status. First, a new figure is created using `plt.figure()` to ensure that the plot is drawn on a fresh canvas.

The `sns.boxplot()` function from the Seaborn library is then used to generate the boxplot. The `x` parameter is set to "Churn" to group the data by churn status, while the `y` parameter is set to "tenure" to show the number of months each customer has been with the company. The `color` parameter is set to 'purple' to give the boxes a consistent and visually appealing color.

A title, "Customer Tenure vs Churn", is added using `plt.title()` to describe what the chart represents. The x-axis is labeled "Churn" and the y-axis is labeled "Tenure (Months)" using `plt.xlabel()` and `plt.ylabel()` respectively, to provide context for the data being visualized.

Finally, `plt.show()` is called to render and display the plot, allowing us to visually compare the tenure of churned versus non-churned customers and identify patterns or differences in customer behavior.

```
In [10]: # Create a new figure for the plot
plt.figure()

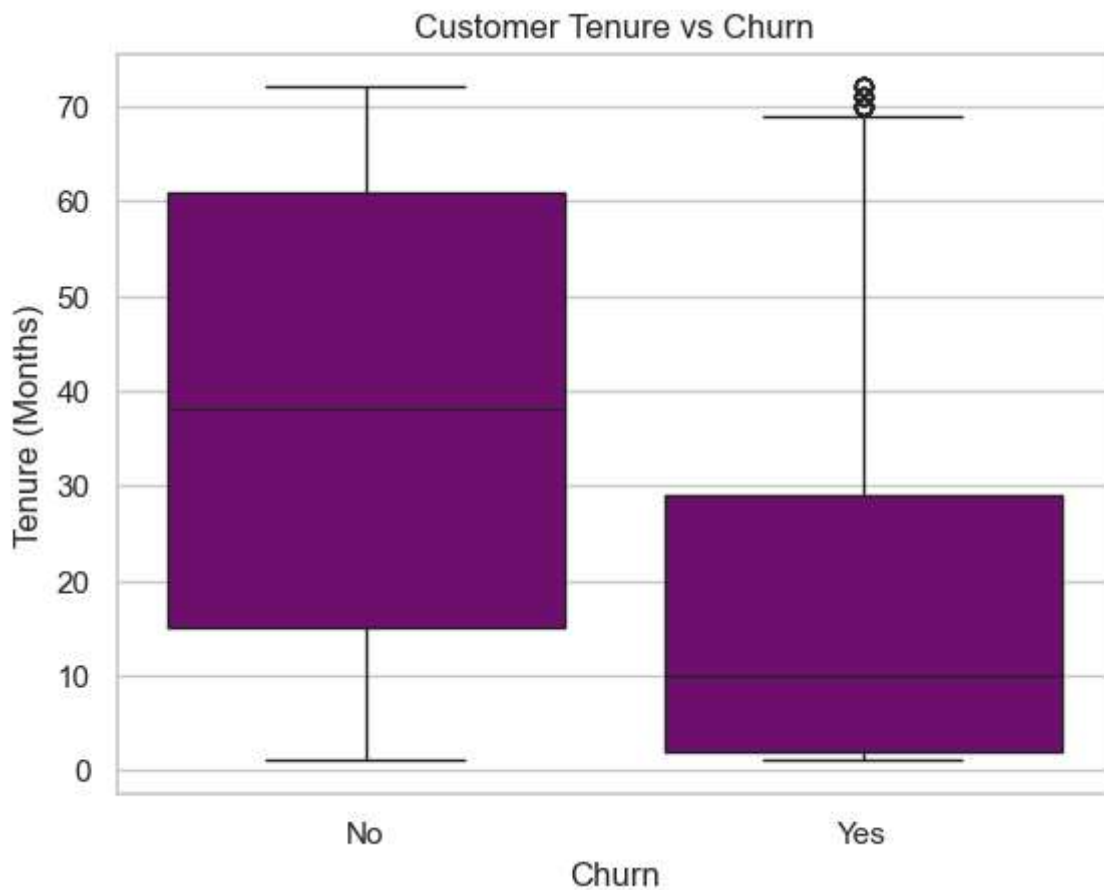
# Draw a box plot to compare customer tenure for churned and non-churned customers
sns.boxplot(x="Churn", y="tenure", data=df, color='purple' )

# Add a title to explain what the chart represents
plt.title("Customer Tenure vs Churn")

# Label the x-axis
plt.xlabel("Churn")

# Label the y-axis
plt.ylabel("Tenure (Months)")

# Display the plot
plt.show()
```



Insight: Customers with shorter tenure are more likely to churn, indicating that early-stage retention is critical.

Graph 3: Boxplot of Monthly Charges by Churn

The following code creates a boxplot to visualize the distribution of `MonthlyCharges` for each `Churn` category in the dataset. First, we create a new figure with `plt.figure()` to ensure the plot is generated in a fresh figure environment.

We then use **Seaborn's boxplot function** to draw the plot, specifying `Churn` as the x-axis and `MonthlyCharges` as the y-axis. The `data` parameter is set to the DataFrame `df`, and the `color` parameter is used to fill all boxes with green, enhancing visual distinction. Boxplots are particularly useful for identifying the median, quartiles, and potential outliers within each category.

Next, we set a descriptive title for the plot using `plt.title("Monthly Charges vs Churn")` to indicate what the visualization represents. We also label the x-axis as "Churn" and the y-axis as "Monthly Charges" using `plt.xlabel()` and `plt.ylabel()`, respectively, to provide clear context for the data.

Finally, we display the completed plot using `plt.show()`, which renders the boxplot with the specified styling and labels. This visualization helps us understand how monthly charges differ between customers who churn and those who do not.

```
In [11]: # Create a new figure for the plot
plt.figure()

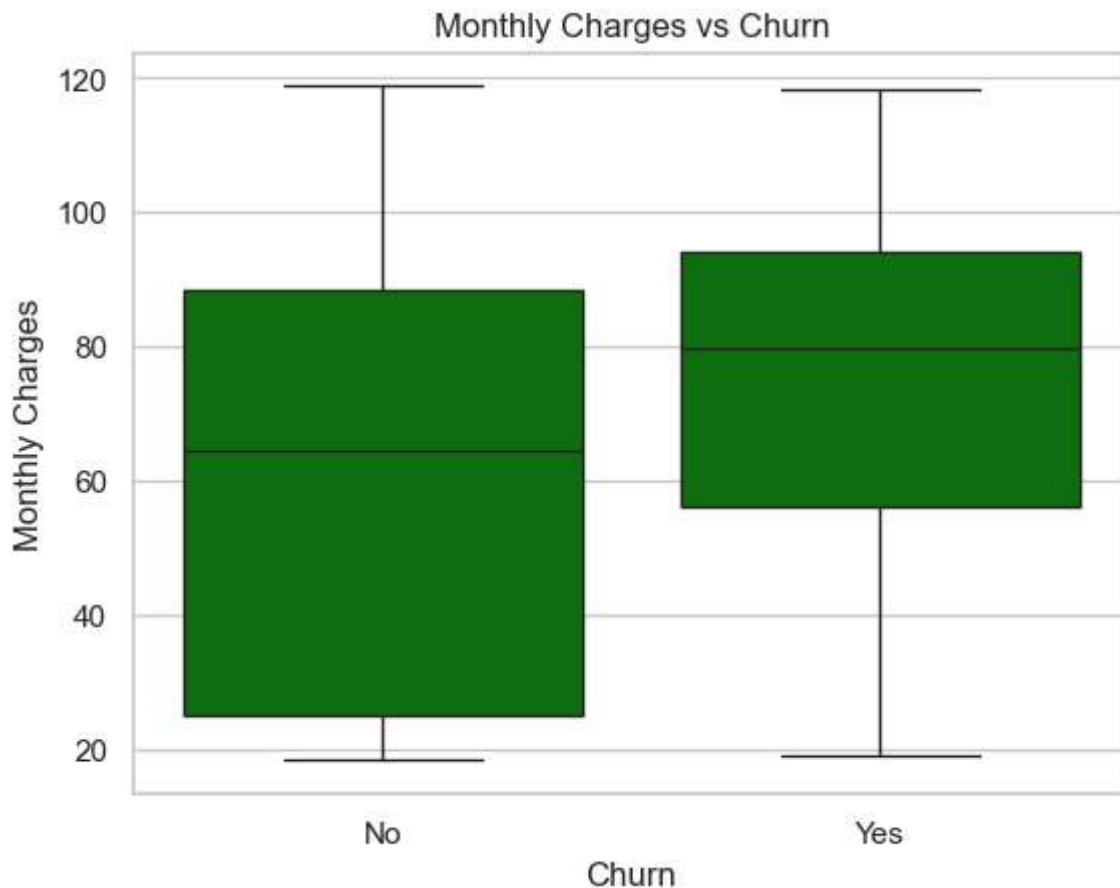
# Draw a boxplot with custom colors for each Churn category
sns.boxplot(
    x="Churn",
    y="MonthlyCharges",
    data=df,
    color='green' # specify colors for each Churn category
)

# Set the title of the plot
plt.title("Monthly Charges vs Churn")

# Label the x-axis
plt.xlabel("Churn")

# Label the y-axis
plt.ylabel("Monthly Charges")

# Display the plot
plt.show()
```



Insight: Customers with higher monthly charges are more likely to churn.

Graph 4: Visualizing Customer Churn by Contract Type

This code creates a visual representation of customer distribution across contract types, with churn status. First, a new figure is created using `plt.figure()` to prepare a clean canvas for the plot.

We then use **Seaborn's `countplot`** to display the number of customers for each contract type. The `hue="Churn"` parameter separates the bars by whether a customer churned, allowing a clear comparison between the two groups. Specific colors are assigned to each churn category using the `palette` parameter, with red representing customers who churned and blue representing those who did not.

To make the plot more informative, a title is added with `plt.title("Contract Type vs Churn")`, and both the x-axis and y-axis are labeled to indicate the contract type and the number of customers, respectively. The x-axis labels are rotated slightly using `plt.xticks(rotation=20)` to improve readability when the labels are long or closely spaced.

Finally, `plt.show()` is called to display the plot, providing a clear visual summary of how customer churn varies by contract type.

```
In [12]: # Create a new figure for the plot
plt.figure()

# Create a count plot showing the number of customers by Contract type
# 'hue="Churn"' separates the bars by churn status (Yes/No)
sns.countplot(
    x="Contract",
    hue="Churn",
    data=df,
    palette={"Yes": "red", "No": "Blue"} # Assign specific colors to each hue
)

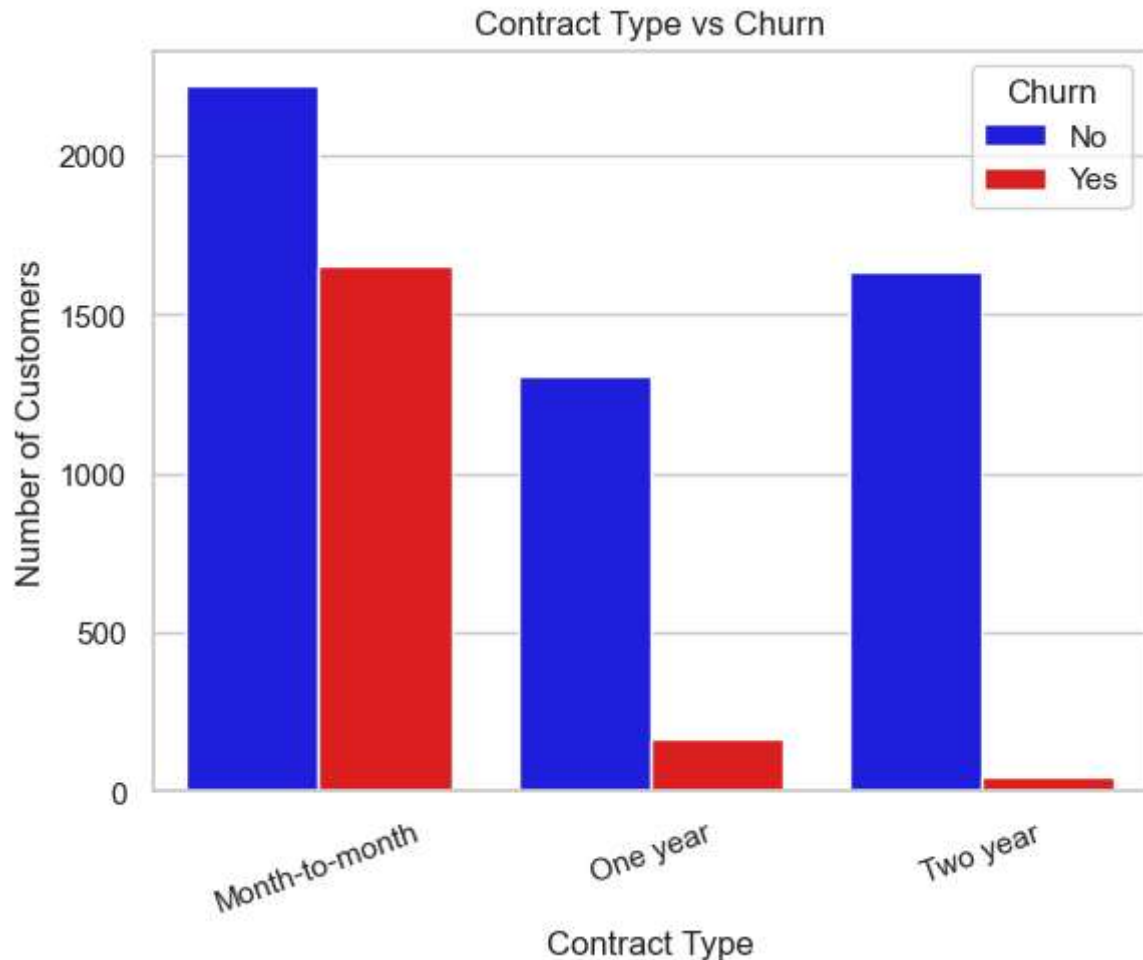
# Add a title to the plot
plt.title("Contract Type vs Churn")

# Label the x-axis
plt.xlabel("Contract Type")

# Label the y-axis
plt.ylabel("Number of Customers")

# Rotate x-axis labels slightly for better readability
plt.xticks(rotation=20)

# Display the plot
plt.show()
```

Insight: Month-to-month contracts have significantly higher churn compared to one-year or two-year contracts.

7. Overview / Conclusion of Graphical Analysis

EDA Conclusion

The exploratory data analysis highlights several important factors influencing customer churn. Customers with short tenure, higher monthly charges, and month-to-month contracts are more likely to leave the company. The churn variable is also imbalanced, which should be considered during model evaluation in later milestones.

These insights confirm that customer churn prediction is a meaningful and relevant business problem. The findings from this graphical analysis provide a strong foundation for feature engineering, model selection, and classification modeling in future stages of the project.

Week 8 - Term Project Milestone 2: Data Preparation

Milestone 2 – Data Preparation for Modeling

Objective

The objective of Milestone 2 is to prepare the dataset for the model-building and evaluation phase by performing essential data preparation steps. This includes selecting relevant features, removing non-informative variables, handling missing data appropriately, transforming variables into appropriate formats, and engineering new features that better capture customer behavior.

Additionally, categorical variables are encoded into numerical representations to ensure compatibility with machine learning algorithms. Throughout this process, care is taken to avoid data snooping by using only information available at the time of prediction, resulting in a clean, reliable, and model-ready dataset.

1. Drop Features That Are Not Useful for Model Building

Explanation

Some features in the dataset do not contribute to predicting customer churn. The **customerID** column is a unique identifier assigned to each customer and does not contain behavioral or demographic information. Including identifier variables in a predictive model can introduce noise and reduce model performance. Therefore, this feature is removed before model development.

```
In [42]: # Drop customerID since it is only an identifier and not useful for prediction
df = df.drop(columns=["customerID"])
df.head()
```

Out[42]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Intern
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

2. Perform Data Extraction / Selection Steps

Explanation

At this stage, a subset of features is selected that are most relevant to predicting customer churn. These features capture customer demographics (e.g., gender, senior citizen status), account information (e.g., tenure, contract type), services subscribed to (e.g., internet, streaming, security), and billing details (e.g., monthly and total charges).

Selecting only relevant variables helps reduce dimensionality, improve model interpretability, and minimize unnecessary complexity. A separate working dataset (`df_selected`) is created to preserve the original dataset and ensure reproducibility of results. No target leakage is introduced during this step, as only existing customer attributes are used.

```
In [43]: # Select relevant columns for churn prediction
selected_columns = [
    "gender",
    "SeniorCitizen",
    "Partner",
    "Dependents",
    "tenure",
    "PhoneService",
    "MultipleLines",
    "InternetService",
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies",
    "Contract",
    "PaperlessBilling",
    "PaymentMethod",
    "MonthlyCharges",
    "TotalCharges",
    "Churn"
]

# Create a working dataset with only selected features
df_selected = df[selected_columns].copy()

# Preview selected data
df_selected.head()
```

Out[43]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Intern
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

3. Transform Features If Necessary

Explanation

Some variables require transformation to be suitable for machine learning algorithms. The target variable **Churn**, which is categorical, is converted into a binary numerical format (Yes = 1, No = 0). This transformation is necessary because most classification algorithms require numerical input. Additionally, numeric conversion of billing-related variables ensures consistent data types for modeling.

```
In [44]: # Convert target variable Churn to binary (Yes = 1, No = 0)

df["Churn"] = (
    df["Churn"]
    .astype(str)
    .str.strip()           # remove spaces
    .str.replace("'", "")  # remove the extra single quotes
    .str.title()           # normalize Yes/No
    .map({"Yes": 1, "No": 0})
)

df.head()
```

Out[44]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Intern
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	



4. Engineer New Useful Features

Explanation

Feature engineering is used to create new variables that may better capture customer behavior. An example is the creation of an **AverageMonthlySpend** feature by dividing **TotalCharges** by **tenure**. This feature provides insight into spending behavior independent of a customer's tenure with the company. This engineered feature is derived solely from existing data and does not introduce future information, thereby avoiding data snooping.

In [45]:

```
# Create a new feature: Average Monthly Spend
df["AverageMonthlySpend"] = df["TotalCharges"] / df["tenure"]

# Replace infinite values caused by tenure = 0 with 0
df["AverageMonthlySpend"] = df["AverageMonthlySpend"].replace([np.inf, -np.inf], 0)

df.head()
```

Out[45]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Intern
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

5 rows × 21 columns



5. Deal With Missing Data

Explanation

The **TotalCharges** variable contains missing values, primarily among customers with very low or zero tenure. These missing values are not random; they represent new customers who have not yet accumulated sufficient billing history.

Instead of dropping these rows, which would reduce the dataset size and potentially bias the model, the missing values are imputed using **MonthlyCharges**. This approach is reasonable because monthly charges closely approximate the expected total charges for new customers. This strategy preserves valuable customer records while maintaining data integrity and consistency.

```
In [46]: # Convert TotalCharges to numeric (coerce invalid values to NaN)
df_selected["TotalCharges"] = pd.to_numeric(df_selected["TotalCharges"], errors="coerce")

# Check number of missing values before handling
df_selected["TotalCharges"].isnull().sum()
```

Out[46]: 11

```
In [47]: # Fill missing TotalCharges using MonthlyCharges for new customers
df_selected["TotalCharges"] = df_selected["TotalCharges"].fillna(
    df_selected["MonthlyCharges"]
)

# Verify missing values are handled
df_selected["TotalCharges"].isnull().sum()
```

Out[47]: 0

6. Create Dummy Variables If Necessary

Explanation

Many features in the dataset are categorical and cannot be directly used by machine learning models. One-hot encoding converts these categorical variables into numeric dummy variables. The first category is dropped (**drop_first=True**) to prevent multicollinearity. This step ensures that the dataset is fully numeric and compatible with common classification algorithms such as Logistic Regression and Decision Trees.

```
In [26]: # Identify categorical and numerical columns
categorical_cols = df.select_dtypes(include=["object"]).columns
numerical_cols = df.select_dtypes(include=["int64", "float64"]).columns

categorical_cols, numerical_cols
```

```
Out[26]: (Index(['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines',
               'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
               'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
               'PaperlessBilling', 'PaymentMethod'],
          dtype='object'),
         Index(['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges', 'Churn',
               'AverageMonthlySpend'],
          dtype='object'))
```

```
In [27]: # Create dummy variables for categorical features
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Preview the transformed dataset
df_encoded.head()
```

```
Out[27]:
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	Churn	AverageMonthlySpend	ge
0	0	1	29.85	29.85	0	29.850000	
1	0	34	56.95	1889.50	0	55.573529	
2	0	2	53.85	108.15	1	54.075000	
3	0	45	42.30	1840.75	0	40.905556	
4	0	2	70.70	151.65	1	75.825000	

5 rows × 32 columns



Conclusion of Data Preparation

In this milestone, the dataset was successfully prepared for the model-building phase through a series of well-justified data preprocessing steps. Non-informative features were

removed, and relevant variables were carefully selected to focus the analysis on factors that influence customer churn. Missing values were handled using a reasoned imputation approach to preserve important customer records and maintain data integrity.

Feature transformations and engineering were applied to improve the representation of customer behavior, while categorical variables were converted into numerical formats suitable for machine learning algorithms. Throughout this process, precautions were taken to avoid data snooping by relying only on existing and appropriate information. As a result, the dataset is now clean, structured, and fully ready for effective model training and evaluation in the next milestone.